

BPMN-LLM

Transforming BPMN Models Into Smart Contracts Using Large Language Models

Sheng Gao¹, Wenting Liu¹, Jianming Zhu¹, School of Information, Central University of Finance and Economics

Xuewen Dong², School of Computer Science & Technology, Xidian University

Jin Dong³, Beijing Academy of Blockchain and Edge Computing

// “Law is code” is pivotal for advancing the intelligent judiciary. This article proposes a business process modeling notation-large language model (BPMN-LLM), which transforms BPMN models of legal contracts (LCs) into smart contracts (SCs) using LLMs in a user-friendly and cost-effective manner. //



©SHUTTERSTOCK.COM/ANDREY SUSLOV

THE CONCEPT OF smart contract (SC)¹ aims to execute predefined terms in a self-enforcing manner. According to a report² of the U.K. Law Commission, a smart legal contract (SLC) is a legally binding contract in which a computer program automates some or all contractual obligations. It features semiautomation and legal enforceability in human-readable and machine-readable paradigms, facilitating cross-domain collaboration with reduced costs for advancing legal digitization. However, how to accurately and efficiently transform a legal contract (LC) into an equivalent SC remains a significant challenge.

Some schemes, such as Caterpillar,³ Lorikeet,⁴ and Pupa⁵ employed the business process modeling notation (BPMN) standard⁶ to model an LC into an SLC specification based on process models and develop a BPMN-to-Solidity compiler to generate an SC in Solidity. However, the compiler relies heavily on manually defined static mapping rules, resulting in the need for programming skills, limited dynamic adaptability, and substantial maintenance costs. It fails to accommodate dynamic changes, making it necessary for skilled programmers to redesign and redevelop mapping rules whenever there are new business requirements or additions of new BPMN elements, leading to high labor costs and prolonged development cycles, ultimately hindering efficiency and scalability.

To lower development barriers and enhance dynamic adaptability, existing works^{7,8,9} have proposed leveraging the understanding and emergent capabilities of large language models (LLMs) to generate SCs. Some works^{7,8} focus on directly transforming texts expressed

in natural language (NL) into SCs. While they can enhance flexibility and user-friendliness, the inherent ambiguity of NL often compromises the quality of the generated SCs. A recent work⁹ leverages ChatGPT to convert the iContractML model into an SC; however, the descriptive prompt paradigm written in NL may struggle to clearly convey complex logical relationships and conditions, affecting the accuracy and reliability of SC generation. In this article, we propose BPMN-LLM to transform BPMN models into SCs by leveraging LLMs with structured chain-of-thought (SCoT)¹⁰ prompting, which has demonstrated superior effectiveness in code generation.¹¹ By extracting information from the SLC specification output by BPMN models to create prompts and applying few-shot learning that harnesses the emergent capabilities of LLMs,¹² our BPMN-LLM can facilitate the generation of high-quality SCs. Our main contributions are summarized as follows.

- *BPMN-LLM framework*: We propose to transform BPMN models into SCs using LLMs, alleviating issues such as high skill requirements, limited dynamic adaptability, and elevated labor costs.
- *Guided prompt engineering technique*: We employ a SCoT to design a task-specific prompt template that guides LLMs in comprehending the structure of a BPMN model and its corresponding SC. By combining in-context learning with a few-shot prompting that leverages the emergent capabilities of LLMs,¹² our BPMN-LLM enhances dynamic adaptability and accuracy in generating SCs.

- *Experimental evaluation*: We present a case study for practical guidance and evaluate BPMN-LLM on three LLMs to generate SCs for four types of LCs. The results indicate that BPMN-LLM ensures compilability and achieves consistency above 97%, demonstrating its effectiveness and efficiency in automating SC generation.

BPMN-LLM Framework

In this section, we present the BPMN-LLM framework as shown in Figure 1, which comprises three stages, providing a comprehensive process for modeling an LC as an SLC and transforming it into an SC.

Stage 1. Formal Modeling

The first stage of the BPMN-LLM framework involves using BPMN⁶ to model LCs and generate SLC specifications, which are then used to design a prompt template that guides the LLMs.

Domain Specification. In general, an LC comprises basic elements and logic structures, where basic elements include contracting parties, objects, rights, and obligations, and logic structures organize contract clauses, including sequential and conditional structures. To facilitate collaboration between legal experts and developers, we employ BPMN that consists of swimlanes, gateways, and flows to model LCs to output SLC specifications. Swimlanes represent contract parties, with pools for whole parties and lanes for task execution. Gateways that include exclusive, inclusive, event-based, and parallel gateways adjust flow paths based on specific conditions. Flows are classified as sequence flows and message flows, illustrating communication within or between pools. Based on BPMN models established for different LCs, we employ the business process modeling language (BPML) to derive semistructured SLC specifications for prompting LLMs.

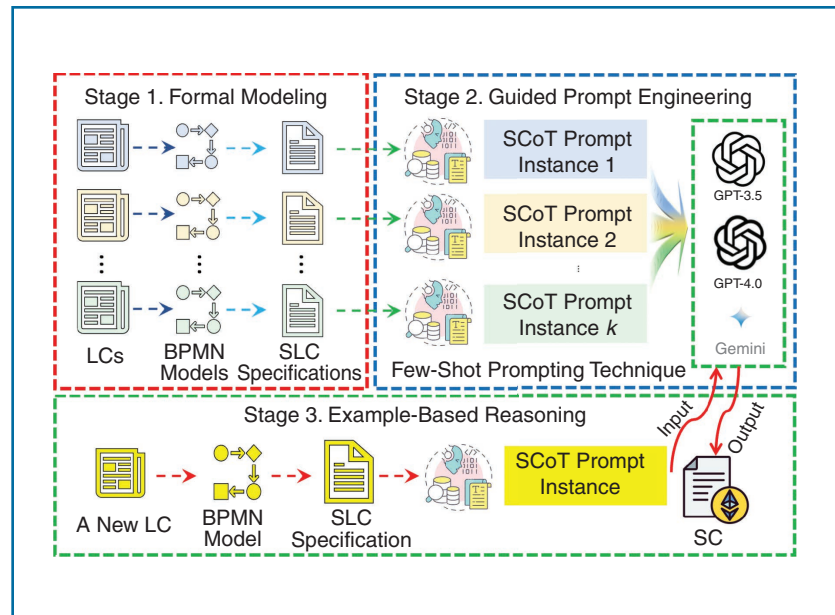


FIGURE 1. The BPMN-LLM framework overview.

Stage 2. Guided Prompt Engineering

In this stage, we tailor SCoT prompts for SLC specification and use few-shot prompting to guide LLMs in transforming BPMN models into SCs.

SCoT. We review prompt engineering to determine an effective prompting technique to guide LLMs in generating specific responses. Chain-of-thought¹³ facilitates LLM interpretation by breaking down complex problems; however, it lacks

structured guidance, struggles with complex logic, and depends heavily on context, limiting its effectiveness in code generation. SCoT enhances the reasoning process by integrating structured guidance and aligning with code structures, enhancing logical stability, reducing context reliance, and boosting SC generation performance. Nevertheless, to our knowledge, no method currently exists for designing prompts tailored to transform SLC specifications into SCs.

Designing a SCoT Prompt Template. Leveraging the structural and logical patterns commonly found within LCs, the SCoT prompt template can be systematically designed for LCs across categories, with adaptations tailored to specific contract types as necessary. As shown in Figure 2, we input a SCoT prompt that includes initialization parameters, events, and functions from an SLC specification and output the corresponding essential elements, including state variables, constructors, events, functions, and modifiers within an SC written in Solidity.

Applying a Few-Shot Learning Technique.

To enhance the dynamic adaptability and generalization, BPMN-LLM leverages the emergent capabilities of LLMs,¹² specifically the few-shot learning to enable the model to learn from only a few well-annotated examples and generalize to new tasks of transforming SLC specifications into SCs. As shown in Figure 3, we present an example of few-shot prompts to describe an instruction, instances, and a new input, illustrating the prompt process of understanding the underlying contextual semantics.

Stage 3. Example-Based Reasoning

For a new BPMN model representing an LC, we input an instantiated SCoT prompt based on the SLC specification into the guided LLM to generate a deployable SC in Solidity and then use the Remix IDE to validate its effectiveness and efficiency.

A Case Study

In this section, we take a sales contract depicted in Figure 4(a) as an example to demonstrate the feasibility of our BPMN-LLM, highlighting the significant value of its practical guidance.

A SCoT Prompt Template

```

{ContractName} Description
initialization parameters:
# These are constructor parameters that initialize the state
variables.
Derive {initialization parameters} from an SLC specification.
events:
# Events are significant moments triggered by conditions or tasks,
influencing the flow of execution within a process.
Derive {eventName} from an SLC specification.
functions:
{onlyRole} do{
  [if {conditions} then:]
    {operationName}() {{operations}};
}
...

The corresponding smart contract in Solidity based on the provided
information about the legal contract.

pragma solidity ^0.8.0;
contract {ContractName} {
  {variable}[access modifier]{initialization parameters};
  ...
  event {eventName}({variable}){initialization parameters};
  ...
  Constructor({variable}){initialization parameters}{{
    initialize the corresponding {initialization parameters};
  }}
  modifier() {onlyRole}(){{
    require({conditions});
  }}
  ...
  function {operationName}() {access modifier} {onlyRole}{{
    [require({conditions});]
    {operations};
  }}
  ...
}

```

FIGURE 2. A SCoT prompt template for an SLC specification.

1. *Modeling an LC using BPMN:* We employ BPMN to model the sales contract, depicted in Figure 4(b). Specifically, the data object contractInfo represents a basic element containing product details and dates, the pools denote the buyer and seller, with lanes allocate their responsibilities. Their rights and obligations are denoted using user tasks such as deliver and refundTo-Buyer of the seller. The gateways model the sales contract logic, adjusting the flows based on conditions. The message flows enable message transfer via message intermediate throw events and continue the process using message intermediate catch events. For example, the deliveryConfirmed emits a message flow with itemDelivered=true to a message intermediate catch events, indicating that the seller has successfully fulfilled the delivery obligation. This triggers an exclusive gateway, which decides whether to proceed with confirmReceipt or requestRefund based on the itemDelivered status.
2. *Generating an SLC specification:* We employ BPML to describe the output of the BPMN model as an SLC specification. Due to the space limits, we only provide a partial segment describing the specification corresponding to the pool of the seller as shown in Figure 4(c), e.g., the <intermediateThrowEvent> corresponds to message intermediate throw event, and the <userTask> corresponds to user task.
3. *Instantiating a SCoT prompt:* According to the designed template, we instantiate a SCoT

prompt with the SLC specification as completed in Figure 5(a). We first extract the basic elements from the SLC specification as initialization parameters for the constructor in the SC. Next, we identify significant

events, such as purchaseConfirmed, deliveryConfirmed, receiptConfirmed, refundRequestIssued, paymentConfirmed, and refundIssued represented as <intermediateThrowEvent> in the SLC specification and instantiate

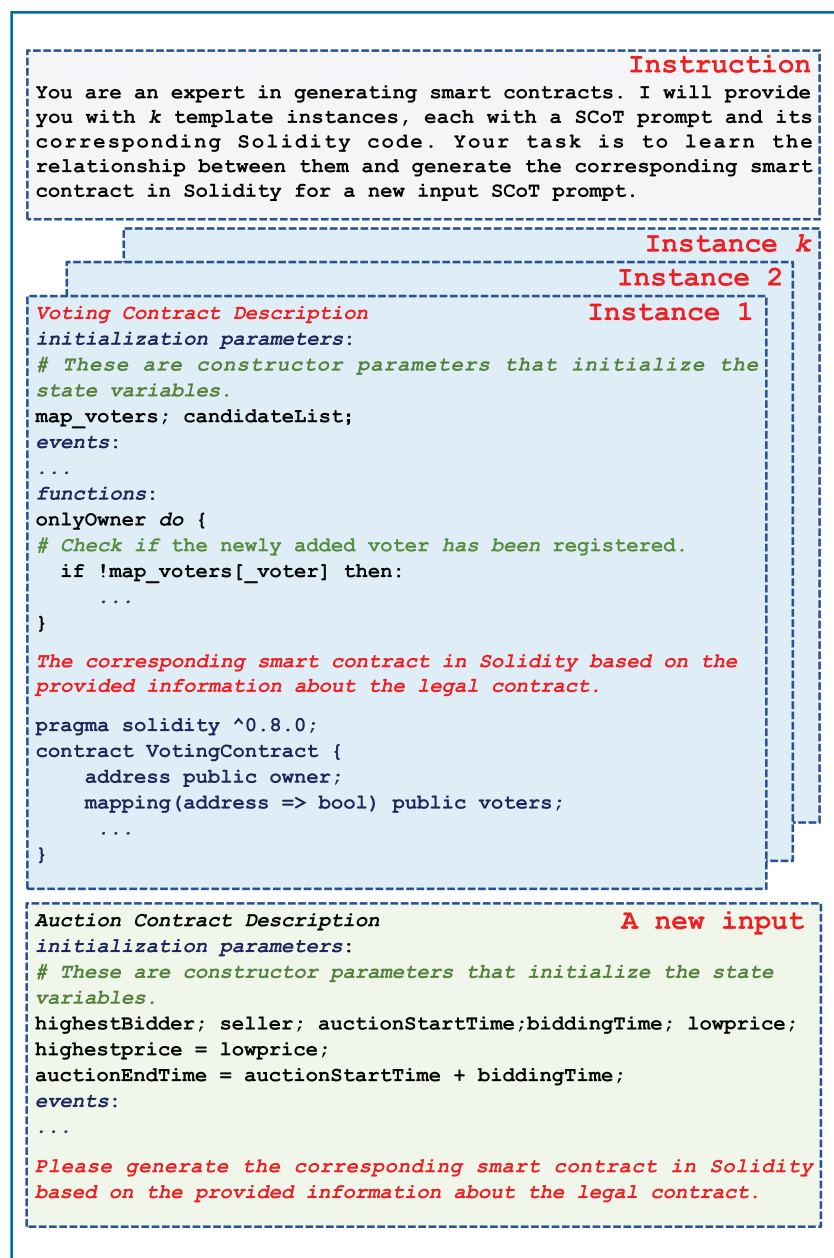


FIGURE 3. An example of few-shot prompts.

them into the events section of the SCoT prompt. Finally, we extract <userTask> elements from the SLC specification, representing the rights and obligations of contract parties,

and instantiate them into the functions section of the SCoT prompt. In addition, we use function modifiers, such as onlySeller and onlyBuyer, to define operational permissions

and specify the authority scope of the buyer and the seller. Note that mastering event emission and state management enables blockchain-based SCs to respond to external events and

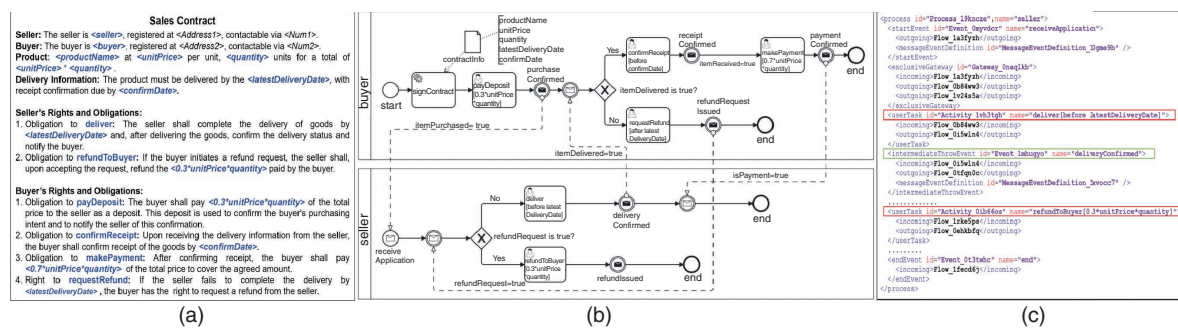


FIGURE 4. The formal modeling process of the sales contract. (a) A sales contract. (b) The BPMN model representing the process of the sales contract in (a). (c) A partial SLC specification generated from the BPMN model in (b).



FIGURE 5. The process of generating the SC in Solidity using GPT-3.5. (a) An instantiated SCoT prompt. (b) The SC in Solidity generated via GPT-3.5.

simulate asynchronous processes through state updates, thereby ensuring compatibility between BPMN and blockchain execution models.

4. *Guide an LLM for SC generation:* Using GPT-3.5 as a sample model, we improve its performance with a few-shot prompting technique for SC generation. Figure 5 illustrates the process from the SCoT prompting, based on the SLC specification of the sales contract, to the generation of the corresponding SC in Solidity. The underlines in the same color indicate the corresponding relationship between the components.

Evaluation

In this section, we present the experimental setup and evaluate the dynamic adaptability of our BPMN-LLM in terms of effectiveness and efficiency.

Experimental Setup

- *Dataset:* We select three types of LCs as few-shot learning examples, including voting, services, and lending, and use four types of LCs to evaluate the performance of our BPMN-LLM across different LC scenarios, including sales, auctions, purchases, and leases. Each LC is BPMN-modeled with 14, 13, 23, and 17 elements, totaling 67.
- *Experimental platform:* The experimental hardware setup consists of dual GPUs RTX4070 Ti with 12GB and an Intel i7-13700KF CPU, running on the Windows 10 operating system.
- *LLMs:* We employ GPT-3.5~ (GPT-3.5 Turbo), GPT-4.0~ (GPT-4.0 Turbo), and

Gemini~(Gemini Pro) for few-shot learning, invoking their APIs to understand prompts and generate SCs in Solidity.

Result Analysis

Effectiveness. Based on a similar principle mechanism, we adopt the benchmarking suite proposed in¹⁴ to evaluate the effectiveness of LLMs in transforming BPMN model elements into target SC elements.

- *Lucidity:* It is calculated by the ratio of the sum of BPMN elements that map to only one corresponding element in the SC to the total number of BPMN elements.
- *Soundness:* It is calculated by the ratio of the sum of BPMN elements that have a mapping relationship with elements in the SC to the total number of BPMN elements.
- *Laconicity:* It is calculated by the ratio of the number of elements in the SC that match only one element in the BPMN model to the total number of elements in the SC.
- *Completeness:* It is calculated by the ratio of the total number of SC elements that have a mapping relationship with BPMN model elements to the total number of elements in the SC.

The effectiveness evaluation results are shown in Table 1. GPT-4.0

and Gemini achieve 100% lucidity, surpassing GPT-3.5's 98.5%. All models score 100% in soundness. Regarding laconicity and completeness, GPT-3.5 and GPT-4.0 slightly outperform Gemini with scores of 98.5%, compared to Gemini's 97.1%. Overall, LLMs excel in transforming BPMN model elements to SC elements in Solidity.

Efficiency. We mainly measure the consistency, compilability, and gas consumption to evaluate the efficiency.

- *Consistency:* It quantifies the degree of alignment between the generated code and its corresponding BPMN process. For each BPMN-modeled LC, a corresponding SC is manually written to accurately reflect the BPMN-defined business process, serving as the ground truth for consistency evaluation. It is calculated as $(LOC-LFC)/LOC$, where LOC represents the total lines of generated code, and LFC represents the lines of modified code necessary to align with the ground truth.
- *Compilability:* It measures whether the generated code conforms to the syntax requirements of the programming language and compiles successfully on the target blockchain

Table 1. The effectiveness evaluation.

Metric	GPT-3.5	GPT-4.0	Gemini
Lucidity	98.5% (66/67)	100% (67/67)	100% (67/67)
Soundness	100% (67/67)	100% (67/67)	100% (67/67)
Laconicity	98.5% (66/67)	98.5% (67/68)	97.1% (66/68)
Completeness	98.5% (66/67)	98.5% (67/68)	97.1% (66/68)

Table 2. The efficiency evaluation.

Metric	GPT-3.5	GPT-4.0	Gemini
LOC	247	251	234
MLOC	248	252	236
bytecode size (bytes)	5,179.5	5,438.25	5,147.75
Consistency	97.9%	99.2%	98.2%
Compilability	100%	100%	100%
Cost (gas)	836,721.3	898,055.3	825,085.3

platform. It is calculated as $(LOC-LMC)/LOC$, where LMC denotes the lines of modified code necessary to ensure successful compilation.

- **Cost:** It refers to gas consumption,¹⁵ which measures the computational resources required to deploy or operate the properly modified SC on the Ethereum blockchain, where $MLOC$ refers to the total number of lines in the SC after modification. We leverage the Remix tool to estimate the gas cost of generated SCs.

The efficiency evaluation results are shown in Table 2. As for compilability, our BPMN-LLM achieves 100% compilability across these LLMs, allowing seamless deployment on the Ethereum blockchain without any modification. As for consistency, GPT-3.5 achieves a 97.9% consistency with five LFC and Gemini improves slightly to 98.2% with four LFC , while GPT-4.0 reaches as high as a 99.2% consistency with just two LFC .

After ensuring consistent modifications, the SCs generated by GPT-4.0 comprise 252 $MLOC$, compiling an average bytecode size of 5,438.25

bytes that requires an average of 898,055.3 gas for Remix deployment. In comparison, the SCs generated by GPT-3.5 involve 248 $MLOC$, with an average bytecode size of 5,179.5 bytes that results in 836,721.3 gas, and that by Gemini is 236 $MLOC$ with 5,147.75 bytes that require 825,085.3 gas. Therefore, a larger SC characterized by increased byte size or LOC demands greater computational and storage resources, resulting in higher gas consumption.

This article proposes the BPMN-LLM framework that integrates BPMN with LLM to facilitate the development of SCs. To improve dynamic adaptability across diverse LCs, we have designed SCoT prompts for SLC specifications and employed a few-shot prompting technique to enhance the performance of LLMs. Experimental results demonstrate the effectiveness and efficiency of our BPMN-LLM in transforming BPMN models into SCs in a cost-effective and user-friendly manner. ☞

Acknowledgment

This work was supported by the National Key Research and

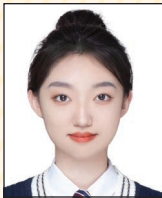
Development Program of China under Grant 2022YFB2701800, the National Natural Science Foundation of China under Grants 62072487 and 62372493, Beijing Natural Science Foundation under Grant L222050, the Fundamental Research Funds for the Central Universities, and Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing. Sheng Gao and Jin Dong are co-corresponding authors.

References

1. N. Szabo, "Formalizing and securing relationships on public networks," *First Monday*, vol. 2, no. 9, pp. 1–15, 1997, doi: [10.5210/fm.v2i9.548](https://doi.org/10.5210/fm.v2i9.548).
2. "Smart legal contracts: Advice to government," The Law Commission, 2021. [Online]. Available: <https://lawcom.gov.uk/project/smart-contracts/>
3. O. López-Pintado, L. García-Bañuelos, M. Dumas, I. Weber, and A. Ponomarev "Caterpillar: A business process execution engine on the ethereum blockchain," *Softw.: Pract. Experience*, vol. 49, no. 7, pp. 1162–1193, 2019, doi: [10.1002/spe.2702](https://doi.org/10.1002/spe.2702).
4. A. B. Tran, Q. Lu, and I. Weber, "Lorikeet: A model-driven engineering tool for blockchain-based business process execution and asset management," in *Proc. BPM Diss. Award, Demonstration, Ind. Track*, Sydney, Australia, 2018, pp. 56–60.
5. R. Naha, and K. Zhang, "Pupa: Smart contracts for BPMN with time-dependent events and inclusive gateways," in *Proc. Int. Conf. Bus. Process Manag.*, 2022, pp. 21–35.
6. M. Geiger, S. Harrer, J. Lenhard, and G. Wirtz, "BPMN 2.0: The state of support and implementation," *Future Gener. Comput. Syst.*, vol. 80, pp. 250–262, Mar. 2018, doi: [10.1016/j.future.2017.01.006](https://doi.org/10.1016/j.future.2017.01.006).
7. E. A. Napoli, F. Barbàra, V. Gatteschi, and C. Schifanella, "Leveraging



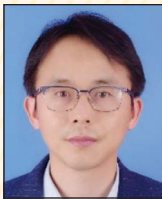
SHENG GAO is a professor at the School of Information, Central University of Finance and Economics, Beijing, 100081 China. His research interests are blockchains and large language models. Gao received his Ph.D. in computer science and technology from Xidian University. Contact him at sgao555@gmail.com.



WENTING LIU is pursuing an M.S. degree in the School of Information, Central University of Finance and Economics, Beijing, 100081 China. Her research is centered on large language models. Liu received her B.S. in information and computing science from Shandong Jianzhu University. Contact her at lwtsucc@163.com.



JIANMING ZHU is a professor at the School of Information, Central University of Finance and Economics, Beijing, 100081 China. His research interests are blockchains and data security. Zhu received his Ph.D. in computer application technology from Xidian University. Contact him at zjm@cufe.edu.cn.



XUEWEN DONG is a professor at the School of Computer Science & Technology, the Engineering Research Center of Blockchain Technology Application and Evaluation, Ministry of Education, and the Shaanxi Key Laboratory of Blockchain and Secure Computing, Xidian University, Xi'an, 710071 China. His research interests are blockchains and network security. Dong received his Ph.D. in computer science and technology from Xidian University. He is a Member of IEEE. Contact him at xwdong@xidian.edu.cn.



JIN DONG is the general director of the Beijing Academy of Blockchain and Edge Computing, Beijing, 100081 China. His research interests are blockchains and artificial intelligence. Dong received his Ph.D. in control theory and control engineering from Tsinghua University. Contact him at dongjin@baec.org.cn.

large language models for automatic smart contract generation,” in *Proc. IEEE 48th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, 2024, pp. 701–710, doi: [10.1109/COMPSAC61105.2024.00100](https://doi.org/10.1109/COMPSAC61105.2024.00100).

8. I. Kang, W. V. Woensel, and O. Seneviratne, “Using large language models for generating smart contracts for health insurance from textual policies,” in *Proc. AAAI Workshop Health Intell.*, 2024, pp. 129–146.
9. N. Petrović and I. Al-Azzoni, “Model-driven smart contract generation leveraging ChatGPT,” in *Proc. Int. Conf. Syst. Eng.*, 2023, pp. 387–396.
10. J. Li, G. Li, Y. Li, and Z. Jin, “Structured chain-of-thought prompting for code generation,” *ACM Trans. Software Eng. Method.*, vol. 34, no. 2, pp. 1–23, 2024.
11. S. Vatsal and H. Dubey, “A survey of prompt engineering methods in large language models for different NLP tasks,” 2024, *arXiv:2407.12994*.
12. J. Wei et al., “Emergent abilities of large language models,” *Trans. Mach. Learn. Res.*, vol. 2022, 2022.
13. J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 24,824–24,837.
14. G. Guizzardi, L. Ferreira Pires, and M. Van Sinderen, “An ontology-based approach for evaluating the domain appropriateness and comprehensibility appropriateness of modeling languages,” in *Proc. Int. Conf. Model Driven Eng. Lang. Syst.*, 2005, pp. 691–705.
15. G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” *Ethereum Project Yellow Paper*, vol. 151, no. 2014, pp. 1–32, 2014.